

Arquitecturas, beneficios, principios claves y cuando considerar su uso

CATEGORIA	ESTILO DE ARQUITECTURA
Comunicación	Arquitectura orientada a servicios (SOA), bus de mensajes
Despliegue	Cliente/Servidor, N-tier, 3-Tier
Dominio	Diseño impulsado por dominio
Estructura	Arquitectura en capas, orientada a objetos y basada en componentes

Arquitectura orientada al servicio (SOA)

Beneficios:

- Alineación de dominio
- Abstracción
- Descubrimiento
- Interoperabilidad
- Racionalización

Principios clave:

- Autonomía
- Servicios distribuibles
- Bajo acoplamiento
- Utilización de esquemas y contratos

Considerar su uso

- Comunicación entre servidor web y servidor de la aplicación
- Cuando se quiera reutilizar servicios
- Se puedan adquirir servicios proporcionados por un host
- Aplicaciones que componen una variedad de servicios en una única interfaz de usuario, o en software S+S, SaaS o basados en la nube.

Arquitectura de bus de mensajes

Beneficios:

- Extensibilidad
- Baja complejidad
- Flexibilidad
- Bajo acoplamiento
- Escalabilidad
- Simplicidad de aplicación

Principios clave:

- Comunicación por mensajes
- Lógica de procesamiento complejo en pequeñas operaciones
- Integración de aplicaciones en diferentes ambientes

Considerar su uso:

- Aplicaciones existentes que interoperan entre sí para realizar tareas
- Combinar varias tareas en una sola operación
- Si se está implementando una tarea que requiere interacción con aplicaciones externas o aplicaciones alojadas en diferentes entornos

Cliente/Servidor

Beneficios:

- Mayor seguridad
- Acceso a datos centralizado
- Facilidad de mantenimiento

Principios clave:

- Comunicación a través de la red
- Concurrencia
- Separación de responsabilidades
- Multiplicidad

Considerar su uso:

- Basado en un servidor que deba responder a muchos clientes
- Aplicación web que será expuesta a través de un navegador
- Implementar procesos de negocio que serán utilizados por toda la organización
- Crear servicios que sean consumidos por otras aplicaciones.
- Centralizar la información, los backup o la administración de funciones.
- La aplicación debe soportar clientes de diferente tipo o diferentes dispositivos

N-Tier/3-Tier

Beneficios:

- Mantenimiento
- Escalabilidad
- Flexibilidad
- Disponibilidad

Principios clave:

- Interdependientes
- Mejora escalabilidad
- Mejora disponibilidad
- Mejora uso de recursos

Considerar su uso:

- El procesamiento de una capa puede afectar el rendimiento de otras capas alojadas en el mismo servidor
- La capa de negocio debe ser montada detrás de un firewall por seguridad, lo que obliga a montar la capa de presentación en una máquina diferente.
- Si se desea compartir la lógica de negocio entre dos aplicaciones y los recursos de hardware lo permiten
- La aplicación a desarrollar posee servidores alojados en una red privada empresarial o en una aplicación de internet donde los requerimientos de seguridad no restringen el despliegue de lógica de negocio en la aplicación del servidor
- Si los requerimientos de seguridad dictaminan que la lógica de negocio no puede estar en el perímetro de la red o la aplicación tiene una exigencia de recursos que precise descargar funcionalidad en otro servidor.

Diseño impulsado por dominio

Beneficios:

- Comunicación
- Eficiente
- Extensible
- Fácil de testear

Principios clave:

- Entender bien el dominio del negocio
- Comunicación al usar misma jerga/lenguaje

Considerar su uso:

- Cuando el dominio de la organización es complejo y se quiere facilitar la comunicación
- Cuando se tenga que diseñar la aplicación en un lenguaje que todos puedan entender
- Cuando los datos a manejar dentro de una empresa son muy grandes y complejos que son difíciles de gestionar por otras técnicas

Arquitectura basada en componentes

Beneficios:

- Facilidad de implementación
- Costo reducido
- Reutilizable
- Mitigación de la complejidad técnica

Principios clave:

- Reutilizable
- Reemplazable
- No es específico del contexto
- Extensible
- Encapsulado
- Independiente

Considerar su uso:

- Si ya se poseen componentes o es posible utilizar componentes de terceros.
- Utilizar componentes diseñados en lenguajes diferentes
- Tener una aplicación con una arquitectura compuesta que le sea fácil reemplazar o actualizar componentes individuales.

Arquitectura orientada a objetos

Beneficios:

- Comprensible
- Reutilizable
- Testeable
- Extensible
- Altamente cohesivo

Principios clave:

- Abstracción
- Composición
- Herencia
- Encapsulamiento
- Polimorfismo
- Desacoplamiento

Considerar su uso:

- Cuando deseas modelar tu aplicación basada en objetos y acciones del mundo real.
- Cuando ya tienes objetos y clases adecuados que coinciden con los requisitos de diseño y operación.
- Cuando necesitas encapsular lógica y datos juntos en componentes reutilizables.

- Cuando tienes lógica empresarial compleja que requiere abstracción y comportamiento dinámico.

Arquitectura en capas

Beneficios:

- Abstracción
- Aislamiento
- Manejabilidad
- Performance
- Reusabilidad
- Testabilidad

Principios clave:

- Abstracción
- Encapsulamiento
- Funcionamiento de capas bien definido
- Alta cohesión
- Reusable
- Bajo acoplamiento

Considerar su uso:

- Separar la lógica de negocios de la lógica de acceso a datos
- Capas existentes adecuadas para la reutilización en otras aplicaciones
- Aplicaciones existentes que exponen procesos de negocio adecuados a través de interfaces de servicio
- La aplicación es compleja y el diseño de alto nivel y exige separación para que los equipos puedan concentrarse en diferentes funcionalidades
- Admitir diferentes tipos de clientes y dispositivos
- Implementar reglas y procesos comerciales complejos y/o configurables
- Mejorar la capacidad de prueba y simplificar el mantenimiento de la funcionalidad de la interfaz de usuario
- Separar el diseño de interfaz de usuario del desarrollo del código lógico que la impulsa
- La vista de la interfaz de usuario no contiene ningún código de procesamiento de solicitudes y no implementa ninguna lógica empresarial